TRUSTSEAL

Complete System Documentation: Product Concepts, Detailed Functional Behaviour, Architecture, Security, User Experience and Engineering Specification

Version 3.0 | 13 November 2026 | Status: Proposed design baseline for review

Source context: TrustSeal UX Design Document and TrustSeal Business Plan, version 1.0 (April 2025). The historical business plan identifies EveryByte Systems, Kampala, Uganda, as the product organisation. Ownership, current implementation status, commercial commitments and approved legal position have not been independently verified.

Preface: purpose, interpretation and document conventions

This document describes TrustSeal as a complete document trust and transaction platform rather than merely a set of screens or a list of functions. Its objective is to provide a shared explanation of the problem being solved, the intended behaviour experienced by each participant, the rules that govern each transaction, the information the platform must retain, and the engineering controls required to make a digital transaction reliable. Product managers should be able to use it to agree scope, designers to derive screens and interactions, software engineers to develop services and interfaces, testers to construct meaningful scenarios, and governance teams to assess operational and legal risks.

The two supplied documents establish TrustSeal's product direction: document upload and drafting, digital stamps, electronic signatures, QR-based verification, contracts, reusable templates, organisations and teams, subscriptions, audit logging, mobile access and integration readiness. They also describe a Flask and PostgreSQL-oriented implementation proposal and an earlier user interface that included rounded controls and gradient treatments. This document preserves the business capabilities but explicitly replaces those visual treatments with the requested restrained, square-cornered enterprise design. All detailed mechanisms, service boundaries, acceptance measures and specialised controls added here are proposed requirements, not statements that the software already possesses those capabilities.

Within this specification, a digital stamp is a visible document mark and an organisational authorisation artefact controlled by TrustSeal. An electronic signature is an action by a person or authorised entity indicating an intention to approve or sign the identified document. A cryptographic digital signature is a technical method using keys and certificates to detect tampering and, subject to trust and identity arrangements, associate a signature with an identity. These are related but not interchangeable. TrustSeal must not promise that an uploaded image, a QR code or a typed name automatically satisfies all applicable legal requirements. Its product experience must explain the assurance method used for each transaction.

1. Product identity and strategic concept

1.1 What TrustSeal is intended to become

TrustSeal is conceived as a secure web-based environment in which organisations and individuals prepare documents, establish who may take action on them, secure approvals, apply controlled stamps and signatures, distribute completed records and subsequently prove that a presented copy corresponds to a registered transaction. The product should replace fragmented workflows involving emailed files,

manually pasted stamp images, unsigned document revisions and informal approval messages with a transaction that can be followed from inception to archival. A successful TrustSeal transaction has an identifiable owner, a definite document version, a documented sequence of decisions, a completed evidence record and an understandable verification result.

TrustSeal should support both simple individual transactions and structured organisational processes without forcing every user through an enterprise-level configuration. A freelance professional might upload an agreement, add a signature field and send it to one client, while a large institution might require legal review, a financial approval, two authorised signers and an official seal whose use is restricted to designated custodians. These workflows should use the same core document, identity, evidence and verification model. Differences should be expressed through permissions, policies, reusable templates and workflow configuration rather than entirely separate products.

1.2 Business model and market positioning

The original business plan presents a subscription product serving individuals, SMEs, legal and finance practices, education, government, NGOs and other organisations. That direction remains coherent for the specification. Subscriptions should govern entitled capabilities, included volumes, storage and service arrangements, but commercial restrictions must never silently compromise the integrity or availability of previously completed evidence. An expired plan may prevent new transactions while allowing appropriately authenticated retrieval and independent verification of completed transactions within published retention terms.

The business plan proposes Free, Pro, Business and Enterprise tiers and gives illustrative historic prices. Those figures are planning assumptions, not a current price catalogue. The platform should implement plan definitions and usage meters as configurable data, so that pricing and commercial packages can change without recompiling core application logic. Enterprise capabilities such as single sign-on, advanced audit export, higher service guarantees, custom domains and dedicated hosting should be governed by explicit entitlements and contract terms.

1.3 Product principles and boundaries

TrustSeal shall prioritise integrity before visual appearance, reliable access control before convenience, and understandable user journeys before feature abundance. A document may be drafted in the system, but TrustSeal is not intended to replace specialised office suites in every advanced authoring scenario. An official stamp can be placed visually, but the placement must also be recorded as a formally authorised system event. A verification page can show that a document fingerprint matches an archived evidence record, but it cannot certify the legal authority of a person unless authority has actually been established through defined checks. These boundaries shall be explicit in the product, technical design and marketing material.

2. Participants, organisational context and authorisation

2.1 Identity, membership and account boundaries

The system shall distinguish a human identity from the organisations to which that person belongs. A user may have a personal workspace, membership in one or more organisations and different responsibilities in each. The organisation is a tenant and is the primary boundary for ownership, confidentiality, configuration and billing. Each stored document, template, stamp, workflow and audit event must carry its owning tenant, with technical enforcement that prevents one organisation from

reading or changing another organisation's information. Membership changes shall not rewrite the historical identity of the person who performed previous actions.

Registration must support email confirmation, secure password enrolment where passwords are used, recovery, session management and configurable multi-factor authentication. Invitations should identify the inviting organisation and intended role; they must expire and must not allow an invitee to choose a stronger role than the one granted. An external signer may be asked to complete a limited transaction without being granted general access to an organisation workspace. The system shall separately record the verified communication endpoint, any additional identity proofing and the authenticated session used during a signing decision.

2.2 Roles, permissions and policy decisions

The legacy UX document names super administrator, organisation administrator, stamper, signer and viewer roles. TrustSeal should extend this into a permission model that supports document creator, document editor, reviewer, approver, sender, stamp custodian, records administrator, auditor, billing manager, integration administrator and platform operator. Roles shall be bundles of explicit permissions, not hard-coded indicators hidden in the user interface. Sensitive operations, including seal issuance, tenant deletion, retention override, bulk export and role elevation, should require higher assurance and, where the organisation policy demands it, approval by another authorised person.

Every server request must evaluate whether the authenticated actor may perform the requested action against the specified organisation, document, state and resource. Hiding a navigation item is useful for usability, but it is not an access-control mechanism. The authorisation decision must also consider contextual policies such as whether the document is still editable, whether the user's approval is prohibited by separation-of-duties rules, and whether a stamp has been suspended. Permissions should be surfaced in understandable terms: when a user cannot apply an official seal, the interface should explain the governing restriction without revealing sensitive administrative detail.

2.3 Delegation and separation of duties

Organisations often delegate authority when a person is absent. TrustSeal shall support time-bounded delegation that identifies the delegator, delegate, scope of authority, permissible transactions and expiry. Delegation must not imply transfer of another person's signature identity. A delegated approver signs or approves in their own identity while the evidence record states the authority under which they acted. A seal custodian may delegate stamp placement only if the organisation's stamp policy explicitly permits delegation. Critical policies should prevent a single actor from creating, approving and finally sealing a high-risk document when independence is required.

3. The document as the central business object

3.1 Document creation, ingestion and validation

A document enters TrustSeal through file upload, template-based creation, document drafting or an authorised integration. The user supplies or confirms its title, category, description, owner, confidentiality classification and relevant parties. The ingestion service validates file type, size, content structure, malware risk and PDF rendering feasibility. It must not trust a filename extension as evidence of the actual format. Unsupported or unsafe files should be rejected with a specific and recoverable message, while the original bytes of accepted documents are stored separately from generated previews and derivatives.

The upload experience should support PDF as the primary transactional format and optionally DOCX or common image types through a controlled conversion pipeline. Conversion can change fonts, pagination, field placement or document semantics; therefore the application must present the rendered version to the initiating user before signature requests are issued. Each imported source is related to a distinct immutable rendition used for the transaction. The system must record the conversion method, processing status and resulting fingerprint so that a subsequent dispute can distinguish the input file from the exact copy on which participants acted.

3.2 Drafts, versions and immutable transaction snapshots

While a document remains in draft, authorised editors may revise its content, metadata and fields. Each meaningful revision creates a new version with its own author, timestamp, change description and hash; earlier versions remain addressable subject to retention policy. Once a version is sent for approvals or signature, the document content and placement instructions become a frozen transaction snapshot. A sender who discovers an error must withdraw or void the running request and create a new version or transaction rather than silently substituting new bytes under an existing signing link.

TrustSeal must distinguish document status from workflow status and from the status of individual participant actions. A document may be archived while its verification evidence remains valid; a signing request may expire while the underlying source document remains a usable draft. Clear state models prevent ambiguous labels such as 'complete' from concealing unsigned parties, unresolved approvals or missing seals. All transitions should have explicit preconditions, the actor responsible, automatic side effects and audit events.

3.3 Classification, search and lifecycle management

Documents should be organised by ownership, client or counterparty, category, tags, associated contract, creation date, expiry date and legal retention class. Full-text search of document contents can materially improve productivity, but it must be scoped to the requester's permissions and must honour classifications that prohibit indexing. Search results must not expose snippets from confidential records to unauthorised users. Users should be able to filter records by document status, counterparties, time windows, outstanding action, template, stamp and responsible department without learning a special query language.

Archiving is a lifecycle action, not deletion. The archive must preserve the final document, transaction manifest, relevant evidence and relationships needed for later verification. Deletion or disposal requires a retention decision, checks for legal hold and dependent records, an auditable approval path where necessary, and secure disposal in the appropriate storage systems. An organisation administrator should not be able to erase mandatory evidence merely to remove an inconvenient action history.

4. Document preparation and authoring

4.1 Document editor and document composition

The proposed editor should handle common business document structures such as paragraphs, numbered clauses, headings, tables, page breaks, variable placeholders and approved branding. The application should provide a reliable preview of final pagination, margins and signature locations. Editing capability must be deliberately constrained so that a document authored in TrustSeal can be rendered consistently; features that cannot be faithfully exported should either be blocked or clearly identified. Autosave should preserve work without implying that the document is signed or published.

The legacy UX concept includes variables such as a client name. Variables should have typed definitions, validation rules, source mappings, visibility rules and default values. A template can bind a variable to a client record or require a human to supply it before a transaction is sent. Missing required variables must be highlighted at preparation time, not after a signer opens the request. Once resolved, variable values and the generated document rendition shall be frozen for the specific transaction so that later updates to a client profile cannot silently modify what was agreed.

4.2 Field assignment and precise placement

A sender should be able to add signature, initial, date, printed name, title, company, checkbox, text entry and stamp fields using a PDF preview. Each field is associated with a designated participant or role, page number, rectangle, required/optional flag and validation rule. Zooming or resizing a browser must not change the stored document coordinates. Placement should use PDF coordinate space or another canonical page geometry, with conversions tested for rotated pages, mixed page sizes and high-resolution screens.

Before dispatch, a preparation validator must confirm that required participants have appropriate assigned actions, that fields do not obscure important document content, that no required signature field is unassigned and that sequencing rules are consistent. A sender should be able to inspect a participant-specific preview that shows only the tasks expected of that participant. Shared fields, conditional fields and read-only acknowledgement fields should be supported only through defined policies rather than ad hoc client-side exceptions.

5. Electronic signing and affirmative consent

5.1 Signing invitations and access to a transaction

A signing transaction is created from an immutable document snapshot and a set of participants. TrustSeal generates single-purpose invitations delivered by email, SMS or an integration. Invitation links shall use unpredictable tokens stored in non-reversible form, have clear expiration and revocation semantics, and be bound to the intended transaction and participant. Opening a link is not itself evidence of signature. Depending on risk, the recipient may need account login, an email or SMS challenge, an identity-provider authentication, knowledge-based checks where lawful, or a higher-assurance external identity verification process.

The recipient interface should be intentionally simpler than the administrative application. A signer sees the document title, sender, participating organisation, required actions, legal disclosure, deadline and a clear path to review the entire document. The signer must be able to navigate pages before consent, inspect all required fields and download a copy if policy allows. The experience must avoid dark patterns such as preselected agreement, hidden conditions, or a button that signs before the actual document is presented.

5.2 Signature capture, meaning and finalisation

The base product can support drawn, typed and uploaded signature appearances, as described in the original UX document. These provide visual representations, not automatically equivalent cryptographic assurance. TrustSeal should separately capture the deliberate act of signing, including the authenticated actor, declared intent, exact document hash, time, consent disclosure version, selected signature method and the relevant verification context. For higher assurance, the platform may integrate with certified trust-

service providers or use certificate-backed signatures under a reviewed key management and identity-proofing model.

Finalisation must be atomic at the transaction level: the system should not announce successful signing before the signer action, evidence record and updated document artefact are durably committed. If document rendering, storage or cryptographic operations fail, the user should receive a recoverable status and the operation should be safely retryable without creating a duplicate signature. Signing evidence should not be editable through the user interface. Corrections require a new transaction or an explicit voiding process, each with its own traceable history.

5.3 Parallel, sequential and conditional signing

Sequential workflows permit the next participant to act only after required earlier participants have completed their step; parallel workflows allow a group of participants to act independently. A transaction may combine these patterns into stages. The workflow engine must resolve who may act now, which prerequisites have been met, and when a stage is complete. If a required participant declines, expires or is replaced, the system should follow an explicit policy: terminate, return to sender, request a substitution or start a corrected transaction. It must never quietly skip a required signer to reach completion.

Where business rules permit signing in a specified order, TrustSeal should record the order in the transaction definition and enforce it on the server. Notifications should reflect real state transitions rather than guesses based on a browser screen. A sender dashboard should show 'waiting for legal review' or 'awaiting second signer' instead of an uninformative 'pending' status. The participant should only see the current step they are authorised to perform, along with a comprehensible indication of what happens afterward.

5.4 Declines, corrections, withdrawals and expiry

A signer needs a safe way to decline and optionally provide a reason. The sender should be informed according to communication policy without exposing unnecessary personal information. A sender may withdraw a request if no legal or organisational restriction prevents it, but withdrawal must not erase signatures that were already recorded; it must create a terminal state that explains why the transaction did not complete. Correcting a recipient's email should require re-verification and invalidate the old invitation. Changing the identity of a signer after signing must require an entirely new authorisation path.

Expiry rules must distinguish an expired invitation from a completed signed record. Retrying notification delivery does not extend expiry unless an authorised actor explicitly chooses to do so. If expiry is extended, the extension should be logged and communicated to affected parties. The system should display dates and time zones clearly, retaining canonical UTC timestamps for evidence and showing local time in the user's preferred zone.

6. Digital stamps and official seals

6.1 Stamp concept, classes and visual design

TrustSeal's stamp feature must distinguish a convenience stamp from an authorised organisational seal. Personal stamps may identify an individual's professional practice; team stamps may mark reviewed or received documents; official organisation stamps may express a formal institutional act with significant operational consequences. A stamp record should include its owner, display name, classification, purpose, permitted users, date behaviour, signature-line behaviour, optional serial number, validity

period, policy and lifecycle state. The uploaded logo or graphic is an asset; permission to use the organisation's seal is a separate controlled entitlement.

A WYSIWYG stamp builder should permit text, logos, borders, date placeholders, authorised signatory lines and consistent physical sizing. The printed result should look predictable when exported to PDF. Stamp appearance may be single-colour or use multiple approved ink colours, but the stamp builder shall preserve uploaded approved logos and text rather than reinterpret branded artwork. The builder must show scale in millimetres or another reliable physical unit, preview sharpness at typical printing resolution and warn when extremely small text will become illegible. Visual changes to an approved company seal must trigger a new design version and approval process.

6.2 Custody, approval and anti-misuse controls

An organisation shall designate custodians and define whether a stamp requires a request, one-person authorisation or maker-checker approval. The custodian grants or denies a specific stamping request against a frozen document hash and a stated business purpose. Permission may be limited by department, document class, counterparty, amount threshold, authorised date range or other organisational controls. The system must not allow a user to bypass custody by downloading an unrestricted stamp image and using it externally as though it were still protected. Where export of a stamp asset is permitted, the system must make the limits of subsequent platform control clear.

Every application of an official stamp must create an evidence entry linking the exact document version, stamp design version, authorising policy, requesting actor, approving actor where applicable, applying actor, placement coordinates, time and resulting document hash. Revoking a stamp design prevents future use but should not retroactively falsify the historical record of documents lawfully stamped while it was active. Suspected compromise must be a separate incident workflow that permits an authorised administrator to flag affected historical transactions and communicate that status through verification.

6.3 Date stamps, signatures and legal meaning

For dated stamps, the date can be a fixed approved date, the system time at execution, or a sender-supplied date subject to policy. Backdating must not be silently accepted as an authentic execution timestamp. The visible date and the technical timestamp are separate properties and should both be recorded. A 'SIGN: .....' line is a place for a signatory mark and is not proof that the person signed. TrustSeal should allow an actual electronic signature field to be coupled to the seal application when a workflow requires both acts.

The stamp service must explicitly explain what its verification attests. A visual stamp alone may communicate internal approval, but it does not by itself establish identity proofing, authority, signer intention or third-party digital certificate validation. Legal statements presented to the user should be reviewed for each supported jurisdiction and assurance level. The product must avoid calling every decorative overlay a legally certified seal.

6.4 Multi-page placement and document integrity

A stamp may be placed once or replicated across selected pages. Multi-page application must preserve each location as an independent placement event or as a fully expanded event set so that a verifier can determine which pages were stamped. The system should support a preview-before-commit step and should reject placement that extends outside the page bounds or invalidates existing digital signatures. Applying a visible mark to a cryptographically signed PDF can invalidate prior signatures if not performed

through a supported incremental update mechanism; therefore the application must either enforce the correct order or use compatible signing technology whose outcome has been validated.

7. Independent verification and evidence

7.1 Verification identifiers, QR codes and public endpoints

Every finalised transaction should receive a random public verification identifier that resolves to a controlled record rather than revealing an internal database key. A QR code can encode a short, HTTPS verification URL with that identifier. Scanning the code opens a responsive verification page showing the record's status, document title where disclosure is permitted, issuing organisation, completion or stamping information and the assurance methods actually performed. The QR code is a locator; it is not itself a cryptographic signature, and a copied QR image cannot prevent another document from displaying the same link.

The public endpoint must minimise personal information, resist enumeration and abuse, and allow organisations to limit public disclosure. A privacy-preserving option may provide only 'registered record found' and a mechanism for the viewer to compare a locally selected file. Public verification must not require a verifier to upload confidential documents unnecessarily; where feasible, client-side hash calculation should compare the file fingerprint without transmitting file contents. Rate limiting, abuse monitoring and clear accessibility support are essential because anyone may reach these endpoints.

7.2 Document fingerprinting and tamper detection

The core integrity mechanism is a canonical cryptographic hash of the exact final document bytes, with an appropriate algorithm such as SHA-256 and a versioned manifest explaining the method. TrustSeal must maintain an immutable association among the final document, participant decisions, stamp operations, timestamps and other evidence. When a verifier uploads a purported original, the service compares the calculated fingerprint to the registered one and explains whether the bytes match. A mismatch does not automatically prove malicious tampering; even benign re-export or metadata changes can produce different bytes. The interface must explain this distinction without claiming broader authenticity than the mechanism supports.

For stronger assurances, the evidence manifest should be sealed using an organisation- or platform-managed signing key whose lifecycle and trust chain are documented. A trusted timestamp authority can be integrated when required by the transaction's assurance profile. Key rotation must preserve the ability to validate historical evidence, while key compromise should trigger a defined incident response and validation warning. TrustSeal shall never describe a database row or ordinary QR code as independently tamper-proof without qualifying the trust assumptions behind its storage and signing system.

7.3 Evidence package and certificate of completion

At the conclusion of a transaction, TrustSeal should assemble a structured evidence package containing immutable document identifiers, source and final rendition fingerprints, participant identities as established, authentication method, consent events, signing action times, IP and device information where lawfully collected, stage transitions, stamp custody approvals, key references and delivery outcomes. A human-readable completion certificate may summarise these events, while the full machine-readable evidence manifest should retain sufficient detail for a subsequent audit or integration. Both records must state their issuer and limits.

The system should make it possible for a duly authorised user to export a final document with its evidence package and to verify the package independently against a documented algorithm. Such portability is important if a customer changes providers or the platform becomes temporarily unavailable. It must also be possible to explain a negative result: a transaction may be valid but not publicly disclosed, formally voided, compromised, expired before completion or simply unknown to the verifier.

8. Contract lifecycle management

8.1 Contract records and the relationship to documents

A contract is a managed business relationship, not merely a PDF with a special label. Its record should capture counterparties, organisational owner, effective date, execution date, renewal date, expiry date, monetary or service obligations, governing jurisdiction, contract classification, confidentiality class and related documents. Multiple documents may be associated with one contract, including drafts, final agreement, amendments, schedules, side letters and termination notice. The signed agreement remains an immutable evidentiary artefact even when the contract record's commercial attributes are later updated.

The contract lifecycle can proceed through request, drafting, internal negotiation, review, approval, signature, execution, active administration, renewal or amendment, termination and archive. Each state should identify the permitted actions and required evidence. A contract can be fully signed yet not effective until a future commencement date or a specified condition is satisfied. Conversely, an expired contract may still have surviving confidentiality or payment obligations. The system must represent these distinctions instead of treating signature completion as the end of the contract workflow.

8.2 Approval routing and negotiation

A contract approval policy should allow routing based on type, value, counterparty risk, department, governing law and requested deviations from an approved template. A legal review may be mandatory for a non-standard indemnity clause; finance approval may be mandatory above a value threshold; an executive signature may be required for specified commitments. Approvers receive a defined task showing the current revision, decision options and relevant history. The system must not carry approvals forward to a materially modified version without an explicit re-approval policy.

Negotiation should retain version lineage and the identity of each contributor. Where redlining is supported, the application should distinguish accepted modifications from comments and proposed text. External collaborators should see only the documents and discussion items shared with them. Confidential internal notes, risk evaluations and internal approval messages must not accidentally appear in the counterparty view or in an exported final contract.

8.3 Obligations, renewals and amendments

Contract administration requires an obligations register that identifies the duty, responsible party, due date or recurrence, trigger event, evidence of fulfilment and escalation rules. Reminder schedules should account for weekends, holidays and time zones where relevant, and should allow owners to acknowledge completion with supporting evidence. Renewal alerts should begin at configurable intervals before notice deadlines, rather than simply on the expiry date. A missed notice deadline should remain visible as an exception until resolved.

Amendments should create a new related transaction that references the original agreement and clearly states what changed. The platform must preserve historical obligations and show whether an amendment

supersedes or merely supplements an existing clause. Termination should retain its authority, notice, effective date and evidence, while keeping prior completed documents verifiable for the applicable retention period.

9. Templates, automation and reusable workflows

9.1 Template governance

A template is a reusable controlled starting point for documents and transactions. It may contain clause blocks, required variables, assigned participant roles, signature and stamp fields, approval routing, notification wording and retention classification. Templates should have ownership, version numbers, approval states and effective dates. An authorised template publisher may release a new version, but active transactions must continue referring to the exact published version from which they originated. Retired templates should no longer appear for new use but remain resolvable for historical evidence.

A template administrator needs a test mode that populates example variables, checks field locations and simulates workflow routing before publication. Required clauses, legal notices and protected fields may be locked from ordinary user editing. Department-specific variations should be supported through governed templates rather than uncontrolled copies that diverge over time. Template usage statistics can help identify failed conversions, commonly changed clauses and unnecessary approval steps.

9.2 Automation rules and controlled intelligence

Automation should initially focus on deterministic processes such as reminders, overdue escalations, delegation expiry, retention alerts and workflow selection based on explicit metadata. Rules must be versioned, tested and explainable; when a workflow routes to a particular approver, the audit record should identify the rule that made that decision. Background jobs require idempotency, retry handling, dead-letter queues and operator visibility so that failures do not create duplicate invitations or repeated signing actions.

The earlier UX document suggests AI-assisted clauses and field recommendations. Such functionality is optional and must be treated as a recommendation layer, not a legal authority or an autonomous signer. AI-generated text must be reviewable and attributable to a human who accepts it. Confidential documents should not be sent to an external model without the tenant's approved data processing controls, contractual basis, retention policy and explicit feature configuration. AI cannot approve legal terms, infer someone's consent or bypass stamp custody.

10. Sharing, messaging and collaboration

10.1 Secure distribution

TrustSeal should distinguish between a document collaboration share, a signer invitation, an approval assignment and a public verification link. These confer different privileges and must be implemented as different types of grants. A user may invite an outside reviewer to comment on a draft without allowing the person to sign, download or forward it. Links should support expiry, revocation, optional recipient verification and access logging. A copied link must not automatically grant indefinite access to every person possessing it.

When a document is downloaded, exported or emailed, the platform loses some technical control over that copy. The interface and policies should say so. Watermarks and download restrictions can deter

misuse but cannot guarantee that a recipient will not retain a copy or create a screenshot. Confidentiality controls must focus on correct permission decisions, encryption, delivery choices, minimal disclosure and enforceable organisational practice.

10.2 Comments, tasks and notifications

Comments should belong to a specific document version and, where appropriate, a page or clause anchor. The system should distinguish internal-only comments from comments visible to external counterparties. Resolving a comment does not necessarily approve a document; approval is a separate explicit decision. Task assignments should expose responsible actor, due date, escalation path and relation to the workflow state. When a document is replaced by a newer version, comments attached to obsolete geometry should remain accessible in version history rather than being misleadingly repositioned.

Notifications must be generated by durable state changes, not solely by user-interface clicks. Delivery channels include email and, where configured, SMS or messaging integrations. Messages should reveal only the minimum document information necessary for the recipient to identify the action. Delivery receipts, bounces, retries and opt-out rules must be retained according to policy. Recipients should be able to view an in-app notification centre without depending on the reliability of their external inbox.

11. Customer and organisation management

11.1 Client and counterparty directory

The client directory should represent organisations and persons with whom a tenant regularly transacts. A counterparty record can maintain names, trusted contact methods, business identifiers, addresses, designated signatories, contract relationships and consented communication preferences. This directory is not a substitute for identity proofing: a saved email address or manually entered company registration number is informational until separately verified. A participant's contact details can be suggested from the directory, but the sender must confirm the intended recipient before dispatch.

Duplicate detection should avoid silently merging people with similar names. Changes to a client's profile must not rewrite the recipient details captured for earlier completed transactions. Directory access should be limited by department or confidentiality classification where a tenant requires that separation. Imports and exports should use validated schemas, preview changes before execution and log any bulk updates.

11.2 Organisation settings and branding

Each organisation should configure its verified name, branding, default document categories, time zone, approved sending domains, notification templates, retention policy, signing assurance profiles, seal governance, roles and plan entitlements. Sensitive changes need an approval path and a recorded effective date. An organisation display name may be edited for typographic reasons, but the verified legal name that appears on assurance records must not be altered without the necessary verification process.

White-label branding may adjust logos, typography and approved colours, but it cannot override security disclosures or remove legally necessary consent information. The system's core interface should remain accessible and predictable across tenants. A custom domain must use verified DNS control and TLS configuration and must not imply that the customer controls platform cryptographic trust material unless a dedicated enterprise architecture actually gives that customer such control.

12. Subscription plans, billing and service entitlements

12.1 Plan and usage management

TrustSeal's commercial model should allow configurable plans, billing periods, seats, included documents, signature requests, official stamp applications, storage volumes, verification allowances and API quotas. Each entitlement requires an unambiguous metering rule. For instance, 'document sent' should be defined as a unique transaction dispatched, not each email reminder; a failed upload should not consume a completed transaction allowance. Customers should be able to inspect usage history and projected limits, and administrators should receive notices before quota exhaustion.

Upgrades may take effect immediately while downgrades may require a future billing date or a grace period, depending on the contract. The platform must not delete historical evidence solely because a subscription ends. Any ability to restrict access to archived records must be consistent with contract terms, privacy duties and required retention. Subscription suspension, payment failure and service cancellation must have separate states so that staff and customers understand the consequence of each.

12.2 Payments, invoices and reconciliation

The billing subsystem should record subscriptions, pricing versions, invoices, adjustments, tax treatment, payments, failed attempts and reconciliation results. Payment card details should be handled by a compliant payment service rather than unnecessarily stored within TrustSeal. In East African markets, an approved integration may support mobile money and local banking rails, but such capabilities are design options subject to partner availability and financial compliance review. A payment callback must be authenticated, deduplicated and reconciled against an internal invoice before entitlements change.

Organisation billing administrators should access invoices and plan controls without gaining rights to confidential contracts. Platform finance staff need tenant-level revenue information without unrestricted access to document contents. Refunds and credit notes should retain their relationship to the original transaction. Billing and product usage events should be exportable to accounting systems through governed integrations.

13. The flat, accessible enterprise interface

13.1 Visual contract and design language

TrustSeal shall employ a flat interface with square corners throughout the application. Border radius is zero for buttons, inputs, cards, tables, dialogs, navigation and menus. Gradients, glass effects, floating decorative chips, tinted highlight pills, oversized metric tiles and gratuitous shadows are prohibited. Depth should be expressed through alignment, consistent spacing, thin neutral rules and clear headings rather than ornament. A restrained palette may use navy or deep blue for primary actions, neutral white and grey surfaces for working areas, red for destructive actions and carefully controlled semantic colours for status text. Status should never rely on colour alone.

The proposed default typography is Arial or a similarly readable system sans serif with an approximately 14-pixel application body size, 13-pixel secondary table text and clearly differentiated headings. Density should be appropriate for business operations: compact enough that users can work with document lists and approval queues efficiently, without shrinking interactive targets or sacrificing legibility. Spacing should follow a small consistent scale, with desktop input heights around 36–40 pixels and touch-friendly

controls on mobile. Design tokens must be shared across the administration console, document management interface, signer portal, verification portal and any embedded integration views.

13.2 Navigation architecture and permissions

The authenticated workspace should use a stable top bar containing the organisation context, global search, notifications, help and account controls, alongside a narrow persistent left navigation on desktop. The main navigation should prioritise Dashboard, Documents, Requests, Contracts, Templates, Stamps and Seals, Clients and Reports. Administration, Billing, Audit and Integrations should appear only for users with appropriate permissions. Users must not be made to browse through disabled modules that they cannot operate. A change of organisation must clearly re-scope every list and search result to prevent accidental cross-tenant action.

Every screen should show a short page title, useful context, a primary action and a clear current location. Lists should use compact tables with meaningful column widths, sort controls, inline filters and a predictable action menu. Important actions should appear directly where they are needed, while infrequently used actions can live in a secondary menu. The navigation structure should remain consistent across modules so that 'New document', 'View details', 'Send for signature' and 'Audit history' behave according to the same interaction patterns.

13.3 Document workspace and transactional editor

The document detail screen should be divided into content and action context, not a cluttered dashboard. A document header provides title, version, owner, status and the permitted next action. Tabs or a secondary navigation can expose Overview, Document, Participants, Workflow, Activity and Evidence, but only when those sections contain relevant information. A preparation view should devote most width to the PDF preview, with a narrow right panel for assigning fields and participants. Users must be able to zoom, navigate pages, undo recent placement actions and inspect validation errors without losing progress.

The signer interface must be especially focused: sender identity and transaction purpose at the top, a large readable document canvas, navigation to required fields, explicit consent and a single unambiguous completion action. The final button must not be enabled until required actions and consent requirements are met. Completion should return a durable receipt and an appropriate download or verification option. Mobile behaviour should use full-width panels and a step progression rather than forcing desktop tables or tiny PDF controls into a narrow viewport.

13.4 Feedback, accessibility and responsive behaviour

All controls shall have text labels or accessible names; focus states should be visible with keyboard navigation; error messages must identify a field and explain how to correct it. Disabled controls should have a discernible reason where appropriate. Loading states should preserve context, prevent accidental duplicate submission and differentiate a slow operation from a failed one. Destructive actions require confirmation explaining the effect, especially for transaction withdrawal, stamp suspension and permanent disposal. Success should be communicated in plain language without excessive banners or celebratory decoration.

The design should meet WCAG 2.2 AA as an engineering target, subject to verified testing, with suitable keyboard access, contrast, screen-reader semantics and reflow. Dates and numerical formats must respect locale and remain unambiguous in evidence records. Responsive behaviour should change layout without hiding essential actions. At smaller widths, tables may become structured records or allow

deliberate horizontal scrolling, while a complex PDF field-placement task may offer an alternative desktop recommendation if touch accuracy cannot be assured.

14. Platform architecture and core services

14.1 Architectural style and transaction boundaries

The source business plan suggests Flask, Jinja2 with JavaScript, PostgreSQL and PDF/image libraries. A modular Flask application remains a valid initial implementation option; there is no need to introduce microservices solely for architectural fashion. The recommended design separates bounded modules for identity, tenants, documents, authoring, workflow, signatures, stamps, verification, contracts, messaging, billing, integrations, reporting and administration. Each module exposes explicit application interfaces and owns its business rules, while a shared transactional database can preserve consistency during the initial product phases.

Long-running work such as malware scanning, document conversion, PDF rendering, certificate application, notification delivery and evidence-package generation must execute through background workers with durable jobs. The web request creates or advances a state and returns a job or transaction identifier; a worker completes the operation and records a final state. Workers must be safe to retry. Commands that change the legal or business status of a transaction require concurrency control so that two simultaneous requests cannot both approve an already finalised action or replace a withdrawn document.

14.2 Storage and database model

PostgreSQL should store transactional entities, relationships, permission decisions and searchable metadata. Object storage should hold original files, derived previews, executed PDFs, stamp assets and evidence packages. The object store needs encryption, access logging, versioning or retention controls appropriate to the assurance level, and a policy that prevents direct unauthorised retrieval through guessed object paths. Database records refer to object identifiers and verified hashes rather than allowing unstructured storage URLs to become public permissions.

Principal entities include Tenant, User, Membership, Role, Permission, Client, Document, DocumentVersion, FileObject, Template, TemplateVersion, Transaction, Participant, WorkflowStage, ActionAssignment, SignatureEvidence, StampDefinition, StampVersion, StampGrant, StampApplication, VerificationRecord, Contract, Obligation, Comment, ShareGrant, Notification, Subscription, Invoice, IntegrationCredential and AuditEvent. The model should use immutable history for completed acts and explicit foreign-key references for relationships. Referential integrity and unique business constraints must be enforced in the database, not merely assumed by application code.

14.3 Hashes, keys and document processing

Document transformations need a controlled pipeline. Each input passes through quarantine, content validation, sanitisation where applicable, rendering, user review, freezing, participant execution and final artifact generation. A finalised document must be content-addressable by its cryptographic digest, and the evidence package must explain which rendering or signing steps altered its bytes. Digital signing keys, if used, must be generated and protected through an approved key-management design, preferably a managed KMS or HSM depending on required assurance. Secrets and private keys must never be committed to source control or exposed in application logs.

The platform must carefully manage PDF incremental updates, embedded signatures, QR placement and stamp overlays because these operations can conflict. Engineering tests should include multi-page files, non-standard page rotations, encrypted PDFs, embedded fonts and documents with pre-existing signatures. Unsupported document features must produce a clear rejection or safe fallback rather than a superficially successful yet invalid result.

14.4 Integration interfaces and event delivery

A versioned REST API should support tenant-scoped document creation, metadata access, transaction initiation, participant status, template retrieval, stamp requests, verification and audit export. API schemas must specify authentication, resource-level authorisation, pagination, search semantics, idempotency keys, predictable error codes and request size limits. Third-party applications should authenticate through short-lived credentials or OAuth-compatible delegated flows where appropriate, with scopes limited to necessary operations. Integrations must never impersonate an individual signer merely by holding an administrator API key.

Webhooks should report durable business events such as transaction.sent, participant.signed, transaction.completed, stamp.applied or contract.expiring. Event deliveries require a stable event ID, tenant ID, schema version, timestamp, signature, retry schedule and replay protection. Consumer outages must not block the primary TrustSeal transaction. API and webhook changes should follow a published compatibility and deprecation policy, with a sandbox where developers can exercise complete scenarios without producing misleading production evidence.

15. Security, privacy and legal assurance

15.1 Threat model and control objectives

TrustSeal's principal risks include account takeover, malicious invitation links, stamp misuse, document substitution, insider data access, forged verification pages, manipulated transaction states, unsafe uploaded files, compromised signing keys and abuse of integration credentials. Security design must consider each actor and trust boundary, including browser, API, background workers, database, object storage, email/SMS providers and external trust services. Strong authentication is necessary but insufficient if document IDs can be enumerated or if server authorisation ignores tenant boundaries.

All traffic must use current TLS configurations, with secure cookies and robust session controls. Sensitive data should be encrypted at rest using managed keys. The service needs appropriate request throttling, CSRF protection for cookie-authenticated actions, safe content-security policies, malware screening, audit trails and anomaly detection. Administrative changes and seal operations should receive enhanced logging and authentication controls. Audit records should be append-only from ordinary application paths, with independent monitoring and integrity protection for higher-risk deployments.

15.2 Privacy-by-design and data protection operations

Personal data processing should be tied to defined purposes, legal bases, retention periods and recipient classes. The platform must explain which information about a signer is collected, whether device or IP context is recorded, how that information is used in evidence, and who may receive it. A privacy request to erase data can conflict with statutory retention or contractual evidence obligations; the system therefore needs an adjudication workflow rather than a simplistic delete button. Access and export requests must be authenticated and appropriately limited to the requester's rights.

Tenant residency and cross-border processing requirements must be evaluated against the jurisdictions and vendors used by a particular deployment. An enterprise customer may require dedicated storage or local hosting, but such commitments must be supported by an actual topology and vendor agreement. Data processing agreements, subprocessor registers, incident reporting processes and support-access policies are operational requirements, not merely text on a privacy page.

15.3 Legal validity and jurisdictional controls

The business plan references Uganda data protection and GDPR readiness. Those statements should be treated as goals requiring legal and operational review, not completed certifications. Electronic signatures and trust services must be assessed for the jurisdiction, document type, identity assurance, consent requirements, evidentiary admissibility and any category-specific formality requirements. TrustSeal should provide configurable assurance profiles, explaining the difference between a basic electronic signature, an authenticated platform signature and a cryptographically verifiable certificate-backed signature.

No user interface copy should claim that every signature or stamp is 'legally binding everywhere' or that TrustSeal independently validates a party's corporate authority without a genuine verification process. The product's terms, evidentiary disclosures, certificate-of-completion wording and retention requirements should receive local counsel review before launch. This documentation specifies engineering controls that can support legal defensibility; it does not constitute a legal opinion.

16. Reliability, scale and operability

16.1 Performance and capacity expectations

The system should define and measure response-time objectives for document lists, search, account administration, previews, upload confirmation, signing and public verification separately. A reasonable initial design target for common authenticated metadata screens is a 95th-percentile server-side response below one second under documented normal load, while larger upload and rendering operations should be asynchronous with clear progress. These are proposed acceptance targets rather than measured product performance. Capacity testing should use representative document sizes, participant counts, concurrent signers, stamp placements and verification traffic.

Database queries must use pagination and appropriate tenant-scoped indexes, while expensive reports should run outside high-priority transaction paths. Object storage and workers can scale independently from the web tier. Quotas must protect shared service quality without blocking access to already complete evidence. As usage grows, module boundaries should support extraction into independent services only where operating metrics justify the extra complexity.

16.2 Availability, backup and disaster recovery

TrustSeal should target an operational service objective defined by the product tier, with monitoring for public verification, document storage, workflow queues, payment integration and signing endpoints. For an initial production tier, a 99.9% monthly availability objective may be used as a design baseline pending commercial approval. The deployment should define recovery point and recovery time objectives and test them through restoration exercises. Backups must include PostgreSQL state, object metadata, stored artefacts, evidence and configuration needed to validate historical records.

A restored environment must not accidentally re-send old invitations or repeat completed stamp and signing operations. Background workers should reconcile durable job states after recovery; integration

callbacks should be deduplicated. Audit evidence and key references require special care: a backup that restores a document without the key history or manifest needed for verification is not a sufficient recovery. Disaster recovery procedures should specify operational ownership, verification tests, communication and post-incident review.

16.3 Observability and support

Operations teams need correlated traces and logs across API requests, document processing jobs, workflow transitions, storage, email delivery and verification. Logs should carry transaction IDs and tenant context while excluding unnecessary document contents, raw invitation tokens and secret credentials. A support console can inspect transaction state and processing errors but should not grant default access to confidential document bytes. Emergency access must be time-limited, justified, approved where practicable and fully audited.

A complete operational model includes incident severity levels, on-call ownership, escalation, maintenance windows, data restoration procedures, dependency health checks, security patch policy and customer communications. Feature flags should enable controlled release of new modules without modifying completed transactions. Releases must include database migration verification and rollback plans that respect the immutability of signed records.

17. Quality assurance, acceptance and traceability

17.1 How requirements become verifiable behaviour

Each described capability must be translated into implementation stories, acceptance tests and evidence of completion. A requirement is not satisfied because a screen exists; it is satisfied when the authorised actor can perform the action, unauthorised actors are blocked, state and data are consistent, failures are recoverable and audit evidence is correct. Product acceptance shall exercise complete journeys, not just individual API responses. Requirements should be versioned and linked to design decisions, test cases and releases in the development repository.

For example, the stamp custody scenario begins when a sender prepares an immutable document and requests a company seal. Tests must show that the sender cannot bypass approval, the designated custodian can inspect the exact requested document, a withdrawn request cannot later be approved, the completed PDF contains the approved visible mark, the audit record identifies the authorising actors and public verification conveys the correct status. This validates governance, documents, workflow, security, PDF rendering and verification as one coherent capability.

17.2 Essential end-to-end acceptance scenarios

The initial acceptance suite should demonstrate a single-party signing request, sequential two-party signing, parallel signing, declined and expired requests, correction by withdrawal and reissue, authenticated stamp approval, stamp suspension, document version preservation, verification of an unaltered final PDF, rejection of a modified file, public verification with limited disclosure, contract renewal reminders, tenant isolation and subscription expiry without evidence loss. Each scenario requires a successful path and deliberate failure path. Concurrency tests should demonstrate that double-clicks, repeated webhooks and simultaneously submitted actions do not create duplicate finalisation.

Security acceptance must include cross-tenant object reference attempts, role elevation attacks, replay of expired links, malicious uploads, forged webhook events, compromised session revocation, audit event protection and backup restoration. Accessibility testing must cover keyboard-only signing and verification

tasks, text alternatives, error focus and mobile reflow. Operational acceptance should include alert generation, worker retry recovery and verification following a simulated disaster recovery exercise.

17.3 Non-functional verification targets

A production readiness decision should require documented evidence of encryption, vulnerability management, retention controls, backup restoration, reliability, accessibility and performance under agreed load. Suggested starting thresholds include WCAG 2.2 AA conformance for supported user journeys, measurable p95 latency targets for metadata reads, no unauthorised tenant access in penetration testing, successful restoration of a sampled completed transaction and independent confirmation that its evidence remains valid. Exact load, latency, RPO and RTO thresholds shall be confirmed against business tier and infrastructure budget before being treated as binding commitments.

18. Phased delivery and implementation sequencing

18.1 Foundation and controlled document transactions

The first implementation phase should establish identity, tenants, secure storage, uploads, document versions, permissions, audit history, PDF preview and a minimal transaction state machine. These capabilities are prerequisites for trustworthy stamping and signing. The initial release should also include safe administrative controls, application monitoring, backup and restoration exercises, and a consistent flat UI shell. A feature should not be considered production-ready merely because the frontend flow is visible while transaction evidence or tenant enforcement remains incomplete.

The second phase should implement signer invitations, field placement, consent capture, evidence packages, visual signatures, controlled stamps, official seal custody and secure public verification. These capabilities should be released together in coherent vertical slices, with their security and document integrity tests. A basic signing feature that lacks durable audit data should be treated as an internal prototype rather than marketed as an evidentiary-grade service.

18.2 Business workflow expansion and enterprise features

The third phase should add contract records, approval policies, template governance, negotiations, obligations, renewals, client directories and broader notifications. These modules are most valuable once the core evidence model and permission framework are stable. Later releases can introduce subscriptions and automated billing, tenant self-service, advanced reports, SSO, API integrations, webhooks and dedicated deployment options. Higher-assurance identity verification and externally trusted cryptographic signature services should be treated as specialised integrations with a separate legal, commercial and operational readiness review.

Throughout delivery, the team should maintain an architectural decision record and a traceability matrix. Any change to document immutability, evidence schema, verification semantics, stamp custody or authorisation rules must receive explicit cross-functional review because these choices have lasting effects on historical transactions. Migration plans should ensure that documents created under earlier assurance versions remain clearly labelled and verifiable after newer features arrive.

19. Engineering contracts and implementation detail

19.1 Document and transaction state machines

The document state model should distinguish DRAFT, PREPARED, IN_PROCESS, FINAL, ARCHIVED and DISPOSED, with additional SUSPENDED or UNDER_REVIEW flags where justified. A draft is mutable; a prepared document has a frozen candidate rendition; an in-process document has active participant tasks; a final document is sealed against content changes; an archived document is retained but not operationally active; a disposed document has passed an authorised retention process. State changes must be conditional on a known version number or equivalent optimistic lock. The system should reject attempts to sign against a stale version rather than attaching consent to unexpected bytes.

A transaction can have CREATED, SENT, ACTIVE, COMPLETED, DECLINED, EXPIRED, WITHDRAWN and FAILED states. These are not interchangeable with participant statuses such as INVITED, OPENED, AUTHENTICATED, COMPLETED or DECLINED. Finalisation should require every mandatory participant action to be valid and every mandatory stamp or approval stage to be complete. The completion service should write a durable finalisation record before reporting success and should ensure any repeated completion request returns the existing result rather than producing a second document.

19.2 Database integrity and event design

Every business entity should have a stable identifier, tenant ownership, creation metadata, last modification information where mutation is allowed, and a schema version where forward compatibility is important. Immutable evidence tables should favour insert-only records with causal references to preceding events. The event record should describe actor, resource, action, resulting state, timestamp, correlation ID, source and authorised context. An event is not a replacement for transactional tables: the platform should maintain a consistent present state for efficient access while preserving sufficient history to explain how that state was reached.

Database constraints should enforce unique invitation tokens, valid workflow stage references, non-overlapping active delegation grants where policy forbids overlap, and document version uniqueness within a document. Object storage commits and database commits cannot be assumed to be one atomic operation; processing should therefore use staged uploads, checksums, durable job states and compensating cleanup procedures. The system should identify orphaned files and incomplete finalisation records through scheduled reconciliation without deleting evidence whose ownership is merely temporarily unresolved.

19.3 API behaviour and failure contracts

An API client creating a transaction should submit the document version, participant assignments, stage definitions and an idempotency key. On success the service returns a stable transaction ID and the current state. Validation errors should identify exact fields and rules, authentication failures should avoid credential leakage, forbidden operations should not expose cross-tenant existence, and concurrency conflicts should identify the resource version to refresh. A retry with the same idempotency key must not send duplicate invitations; a retry with conflicting request content under the same key should be rejected.

A verification API should separate public metadata from private evidence and full document retrieval. Public access should normally return a minimal record status and a document matching mechanism. An authenticated tenant auditor may retrieve a detailed evidence manifest according to policy. API

responses should identify their assurance profile and evidence schema version so downstream systems do not infer stronger guarantees from a generic boolean field such as 'verified'.

20. Governance, open decisions and sign-off

20.1 Decisions required before a binding development baseline

The product owner must confirm the current legal owning entity, brand control, target jurisdictions, primary customer segment, whether personal accounts and multi-tenant enterprises launch together, the assurance levels to be offered, and the legal interpretation of official stamps. Engineering leadership must confirm the deployment model, trust-service partners, supported PDF feature set, storage and encryption approach, SSO roadmap and operational service targets. The finance function must confirm price plans and payment integrations. The design team should approve the flat interface tokens and ensure that every module, not only the marketing site, follows them consistently.

These are not reasons to postpone core design work: the model above defines a coherent recommended baseline. However, any legal assertion, assurance badge, customer-facing service commitment or regulated integration must be held until the relevant owner has reviewed the claim and its supporting technical evidence. Historical milestones, budget assumptions and competitor comparisons in the April 2025 business plan should not be presented as current facts without refreshed research.

20.2 Definition of completion

TrustSeal system documentation is complete when its descriptions can be translated into designs and tests without relying on guesswork about identity, authority, transaction state, evidentiary meaning or document version. TrustSeal implementation is complete for a capability only when business actors can execute its intended journey, security boundaries are enforced, failures do not corrupt the transaction, completed records remain independently verifiable to the stated assurance level, and operating teams can support and recover the feature in production. This distinction between a comprehensive proposed design and verified implemented functionality should govern all future documentation and release communications.

Appendix A. Recommended functional requirement interpretation

Rather than restating the platform as hundreds of disconnected bullet-point statements, functional requirements in this document are expressed through the binding behaviours described in Chapters 2 through 19. Each 'shall', 'must' or equivalent mandatory statement should become a distinct traced implementation requirement during backlog elaboration. The functional domains are identity and tenancy, organisational governance, document storage and versioning, authoring and templates, participant workflows, electronic signatures, stamps and seals, integrity verification, contracts, collaboration, client records, billing, integrations, administration and support. The acceptance model in Chapter 17 specifies how completion must be demonstrated across these domains.

Appendix B. Recommended non-functional requirement interpretation

Non-functional obligations are intentionally integrated into the feature descriptions because they constrain each business action. Security and tenant isolation apply to all document endpoints; integrity and immutability apply to frozen and completed artefacts; reliability and idempotency apply to all externally triggered operations; accessibility applies to signers and administrators alike; and recovery

must preserve historic verifiability. Chapter 16 provides an initial operational baseline and Chapter 17 defines the evidence required before claims of compliance, performance or service readiness may be approved. Numerical targets identified as proposed must be finalised in the non-functional test plan rather than assumed to be contractual commitments.

Appendix C. Source and design provenance

This narrative is grounded in the supplied 'TrustSeal UX Design Document' and 'TrustSeal – Business Plan', version 1.0, dated April 2025. Source-derived capabilities include document upload, rich drafting, drag-and-drop fields, signature invitations, drawn/typed/uploaded signature appearances, stamp design and placement, QR verification, contract workflows, client records, teams, templates, audit views, notifications, mobile responsiveness, subscriptions, APIs and the indicative Flask/PostgreSQL technology direction. The material expands those concepts with proposed integrity controls, identity assurance, stamp custody, evidence architecture, tenant isolation, structured state models, contract obligations, secure APIs, operational procedures, accessibility targets and assurance-specific governance. The original document's rounded/gradient-oriented visual suggestions are deliberately superseded by the requested flat, square-cornered design language.